

■ AI Resume Scanner

NLP-Powered Resume Screening · Semantic Embeddings · Streamlit

Aman Shingane · github.com/AmanShingane/AI-Resume-Scanner

1. Project Overview

The **AI Resume Scanner** is an end-to-end resume screening system that uses natural language processing to semantically compare candidate resumes against a job description. Unlike keyword-based systems, it captures the *meaning* of text using dense vector embeddings, resulting in much more accurate matching. The application accepts multiple PDF resumes in bulk, ranks all candidates by their match score, and extracts skills — all through a clean Streamlit interface.

2. Key Features

Feature	Description
Bulk PDF Upload	Screen multiple resumes simultaneously
Semantic Matching	Sentence-Transformer embeddings capture meaning, not just keywords
Cosine Similarity Score	Each resume receives a numerical match score (0–1)
Skill Extraction	Key technical & soft skills auto-extracted from resume text
Ranked Leaderboard	Candidates sorted highest-to-lowest match score via pandas
Cloud Deployable	Runs on Streamlit Community Cloud with packages.txt config

3. How It Works — Processing Pipeline

The system follows a 5-step NLP pipeline for each uploaded resume:

1	Parse PDF pdfplumber / PyMuPDF extracts raw text from each uploaded PDF resume.
2	Preprocess Text NLTK cleans both the resume and the job description — lowercasing, stopwords removal, lemmatisation.
3	Generate Embeddings sentence-transformers converts the cleaned text into a high-dimensional dense vector (semantic embedding).

4

Compute Similarity

scikit-learn calculates cosine similarity between the JD vector and each resume vector, yielding a score from 0 to 1.

5

Extract & Rank

Skills are extracted from resume text, and all candidates are sorted by match score using pandas.

4. Source Modules (src/)

Module	File	Responsibility
PDF Parser	<code>src/parser.py</code>	Extracts raw text from PDF resumes using pdfplumber/PyMuPDF
Text Preprocessor	<code>src/preprocess.py</code>	Tokenises, removes stopwords, lemmatises text via NLTK
Vectorizer	<code>src/vectorizer.py</code>	Generates sentence embeddings via sentence-transformers
Matcher	<code>src/matcher.py</code>	Computes cosine similarity between JD and resume vectors
Skill Extractor	<code>src/skill_extractor.py</code>	Identifies and returns a list of skills from resume text

5. Technology Stack

Category	Library / Tool	Version	Purpose
Web Framework	Streamlit	1.45.1	Interactive UI & deployment
NLP Embeddings	sentence-transformers	2.7.0	Semantic vector generation
Deep Learning	PyTorch	2.1.2	Backend for transformers
ML Utilities	scikit-learn	1.5.2	Cosine similarity calculation
Text Processing	NLTK	3.9.1	Tokenisation & lemmatisation
PDF Parsing	pdfplumber	0.11.6	PDF text extraction
PDF Parsing	PyMuPDF	1.26.0	Alternative PDF extraction
Data Handling	pandas / numpy	2.2.3 / 1.26.4	Data manipulation & arrays

6. Run Locally

Clone the repository

```
git clone https://github.com/AmanShingane/AI-Resume-Scanner.git
cd AI-Resume-Scanner
```

Install dependencies

```
pip install -r requirements.txt
```

Launch the app

```
streamlit run app.py
```

The app opens at **http://localhost:8501**. On first run, sentence-transformers will download the embedding model (~90 MB) automatically.

7. Streamlit Cloud Deployment

The repo includes all necessary config files for one-click deployment on Streamlit Community Cloud:

File	Purpose
<code>requirements.txt</code>	Python package list — installed automatically by Streamlit Cloud
<code>packages.txt</code>	System-level apt packages (e.g. poppler-utils for PDF support)
<code>runtime.txt</code>	Pins the Python version used in the cloud environment

Steps: Fork the repo → go to share.streamlit.io → New app → select repo → set app.py → Deploy.

GitHub: github.com/AmanShingane/AI-Resume-Scanner